Software Design Specification Document

Export Edge AI

By

Muhammad Moosa – 405243

Hannan Yousaf Butt – 405326

Muhammad Ammar bin Akram – 414563

Supervisor: Dr. Shahzad Younis

Co-Supervisor: Dr. Momina Moetesum

Bachelor of Science in Computer Science (2022 - 2026)

Department of Computing

School of Electrical Engineering and Computer Science

National University of Sciences & Technology (NUST)

Table of Contents

List of Diagrams 4
1. **Introduction** 7
1.1. Scope 7
1.2. Key Objectives 7
1.3. Modules Developed 8
1.3.1. Frontend Interface 8
1.3.2. Computer Vision Module 8
1.3.3. Language Module (LLM QA) 8
2. Design Methodology and Software Process Model 8
2.1. Design Methodology 9
2.1.1. Simplicity 9
2.1.2. Modularity 9
2.1.3. Scalability 9
2.1.4. Flexibility 9
2.2. Software Process Model 9
2.2.1. Why Incremental and Iterative? 9
2.2.2. Stages in Incremental-Iterative Model for ExportEdge 9
2.2.3. Practices Followed in Process Model 10
3. System Overview 10
3.1. Functionality and Context 10
3.2. Architectural Design 11
3.2.1. Layer 1: Data Acquisition & Persistence Layer 12
3.2.2. Layer 2: AI Analysis (Processing Layer) 13
3.2.3. Layer 3: Intelligence Layer 13
3.2.4. Layer 4: Frontend Layer 13
3.3. Modular Collaboration 13
3.4. Decomposition Rationale 13
4. Design Models 15
4.1. Use Case Diagram 15
4.2. Sequence Diagrams 16
4.2.1. Sequence Diagram: Automated Mango Processing Pipeline 16

4.2.2. Sequence Diagram: Frontend Report Retrieval 17
4.2.3. Sequence Diagram: User Query and LLM Response 17
4.3. Activity Diagrams 18
4.3.1. Activity Diagram: Real-time Mango processing and Sorting 18
4.3.2. Activity Diagram: User Query Answering 19
4.4. Data Flow Diagrams 20
4.4.1. Data Flow Diagram Level 0: Context Diagram 20
4.4.2. Data Flow Diagram Level 0: High Level System Decomposition 21
4.4.3. Level 1 DFD: Analyze Mango Quality 23
4.4.4. Level 1 DFD: Orchestrate and Generate Report 23
4.5. State Transition Diagrams 24
4.5.1. Packhouse Sorting Controller State Machine 24
4.5.2. Intelligence Orchestrator State Machine 26
4.6. Class Diagram 28
5. Data Design 29
5.1. Overview of Data Organization 29
5.2. Data Lifecycle in ExportEdge AI 29
5.3. Major Data Entities and Storage 30
5.3.1. Temporary File System Storage 30
5.3.2. Database Storage 30
5.4. Database Entities and Data Representation 30
5.4.1. ExportReport Entity 30
5.4.2. ExportReportItem Entity 30
5.5. Entity Relationship Diagram 31
5.6. Data Dictionary 31
5.6.1. ExportReport Entity 31
5.6.2. ExportReportItem Entity 32
5.6.3. Entity Relationships 32
6. User Interface Design 32
6.1. Screen Images 33
6.1.1. Landing Page 33
6.1.2. Conversational Dashboard (Core functionality) 33
6.2. Screen Objects and Actions 33

List of Diagrams

Figure 1: Box-and-Line Diagram of the Overall System Architecture 11
Figure 2: Architecture Diagram for ExportEdgeAI 12
Figure 3: Use Case Diagram for ExportEdgeAI 15
Figure 4: Sequence Diagram for Mango Processing Pipeline 16
Figure 5: Sequence Diagram for Frontend Report Retrieval 17
Figure 6: Sequence Diagram for user QA 18
Figure 7: Activity Diagram for Mango Processing & Sorting 19
Figure 8: Activity Diagram for User QA 20
Figure 9: Level 0 (context diagram) Data Flow Diagram for ExportEdge AI 21
Figure 10: Level 0 Data Flow Diagram for ExportEdge AI 22
Figure 11: Level 1 Data Flow Diagram for Analyze Mango Quality Process 23
Figure 12: Level 1 Data Flow Diagram for Orchestrate & Generate Report Process 24
Figure 13: State Machine Diagram for Packhouse Sorting controller 25
Figure 14: State Machine Diagram for Intelligence Orchestrator 27
Figure 15: Class Diagram for ExportEdge AI 28
Figure 16: Entity Relationship Diagram for ExportReport Database 31

Revision History

Name	Date	Reason for Changes	Version

Application Evaluation History

Comment (by committee) *includes the one given at scope time both in the document and presentation	**Action Taken**

Supervised by:

Dr. Shahzad Younis

Signature:

1. Introduction

The project, **Export Edge AI**, is designed as an integrated agri-tech solution aimed at transforming the mango exporting business by reducing post-harvest loss and enhancing market competitiveness. This system leverages deep learning (CNNs) and Large Language Models (LLMs) with Retrieval-Augmented Generation (RAG) to perform real-time quality control and market intelligence. By accurately classifying mangoes and providing instant export insights, Export Edge AI automatically generates comprehensive reports, optimizing pricing and destination recommendations for producers and exporters.

1.1. Scope

The **Export Edge AI** platform is built to provide an integrated agri-tech solution, leveraging computer vision and real-time market intelligence, to maximize profits and reduce post-harvest losses in the mango exporting business. Its applications are essential across the supply chain:

- **Real-time Quality Control:** The system captures high-quality images of mangoes on a conveyor belt using a camera. It automatically analyzes each mango using deep learning models to classify it as healthy or diseased
- **Disease Analysis (Segmentation):** For all mangoes, the system segments the affected regions to determine the extent and area of any defects or irregularities accurately, whether the mango is healthy or diseased.
- **Export Intelligence:** The classified mango data is sent to a Large Language Model (LLM) which, using Retrieval-Augmented Generation (RAG), predicts the most suitable export country and optimal marketplace
- **Data Export and Reporting:** Users can generate and export detailed analysis reports produced by the LLM in various formats (e.g., PDF, DOCX) for documentation and future reference.
- **Edge Processing:** The classification and segmentation models are deployed on edge devices (e.g., Raspberry Pi) for running locally.

1.2. Key Objectives

The key objectives of ExportEdge AI include:

- **Real-Time Quality Assurance:** To develop and deploy a system on edge devices capable of classifying mango images (healthy vs. diseased) within **1 second** and generating segmentation masks for defected regions within **2 seconds**, thereby enabling fast, automated sorting and reducing post-harvest loss.
- **Predictive Market Intelligence:** To leverage an LLM with RAG and integrate with external Market Prices APIs to accurately predict the optimal export price (with a Mean Absolute Error (MAE) of less than **10%**) and the most suitable destination country.
- **Scalable Edge Deployment:** To design a system where core AI models (classification and segmentation) are deployed locally on limited resource edge hardware (e.g., Raspberry Pi) while ensuring high reliability (98% uptime) and continuous operation for at least 7 hours without memory overflow.
- **Data Integrity and Security:** To ensure secure storage of all image data, classification results, and price predictions to prevent unauthorized access or modification, and comply with data privacy and institutional security policies, enforcing HTTPS for all data exchanges.

- **Modularity and Reusability:** To engineer a modular codebase with clear documentation and separation of concerns, specifically ensuring that the classification and segmentation components can be reused for analyzing similar agricultural products.

1.3. Modules Developed

The Exportedge AI platform is comprised of the following key modules:

1.3.1. Frontend Interface

This module functions as the **User Interaction Layer**. It is currently in development and provides an intuitive web interface for users:

- A screen that displays the **real-time video** from the camera and shows the **classification result** of mango images.
- An interface with a **prompt bar** for users to write queries for the LLM.
- A section allowing the user to **upload images** of mangoes to be passed to the LLM.
- The interface is designed to be responsive for compatibility across **desktops and tablets**.

1.3.2. Computer Vision Module

This is the **Deep Learning Models** component responsible for real-time image analysis. This module, which includes classification and segmentation, is currently **completed**.

- It uses a Deep Learning framework, specifically **TensorFlow (Version 2.16+)** for the Mobilenet classification model and **PyTorch (version 2.8+)** for the deeplab v3 segmentation model.
- It handles the **Mango Image Classification** (a high-priority feature) to detect disease and classify mangoes.
- It performs **Mango Image Segmentation** (a medium-priority feature) to segment affected regions in mangoes and produces segmentation masks.
- **Operation:** The system must capture the frame when a mango appears, preprocess it using libraries like **OpenCV (version 4.10+)**, and run the models sequentially.

1.3.3. Language Module (LLM QA)

This component integrates the computer vision output with market intelligence to generate export advice. This module and its integration with the market API are currently **in development**.

- It utilizes a Large Language Model (LLM) which employs **Retrieval-Augmented Generation (RAG)** to provide insights.
- The LLM responds to user queries, predicts the **most suitable export country**, and recommends the **optimal marketplace**.
- **Data Integration:** It integrates with external **Market Prices APIs** to access market-specific data necessary for price prediction.
- **Output:** It generates a detailed **analysis report** for the mangoes processed, which users can then download.

2. Design Methodology and Software Process Model

This project adopts a **modular object-oriented design methodology**, combined with an **incremental and iterative software process model**. This combination was selected to effectively manage the complexity of

a multi-stage AI-driven system involving image-based disease detection, segmentation, regulatory compliance analysis, and export decision-making.

2.1. Design Methodology

The system is designed using an **Object-Oriented Programming (OOP) approach**, implemented in a modular manner. Each major functionality of the system is encapsulated within a dedicated module or class, enabling separation of concerns, scalability, and maintainability.

2.1.1. Simplicity

The object-oriented modular design simplifies system development by dividing the overall workflow into clearly defined components such as mango image classification, disease segmentation, severity analysis, regulatory compliance checking, and export recommendation. Each component is implemented as a self-contained module, making the system easier to understand, develop, and debug.

2.1.2. Modularity

Each module performs a specific, independent function (e.g. classification module, decision module), allowing focused development and testing of each component and simplifies future enhancements or replacements without affecting the entire system.

2.1.3. Scalability

The modular design enables the system to scale efficiently. New disease classes, improved machine learning models can be incorporated by extending existing classes or adding new modules, without requiring major changes to the core system architecture.

2.1.4. Flexibility

The object-oriented design provides flexibility in experimentation and deployment. Each module can be refined and adjusted to accommodate new requirements or improve performance under iterative model.

2.2. Software Process Model

The project follows incremental and iterative software development model, which emphasizes iterative refinement and incremental addition of features in the project.

2.2.1. Why Incremental and Iterative?

- **Iterative Development:** Machine learning models require repeated training, evaluation, and refinement. The iterative model supports continuous improvement of classification and segmentation performance.
- **Incremental Integration:** More features can be added in the system while further experimentation of old features continues.
- **Adaptability:** The chosen process model allows new functional requirements to be incorporated in future iterations without restructuring the existing system.
- **Early Validation:** Each iteration results in a functional prototype, enabling early validation of each module before extending the system further.

2.2.2. Stages in Incremental-Iterative Model for ExportEdge

- **Requirement Analysis and Planning:** Identification of core system requirements such as mango image classification and disease segmentation and export intelligence.

- **Prototyping**: Design and implementation of mango disease classification model, followed by the development of a segmentation model to localize defective regions in mangoes.
- **Incremental Module Implementation**: Implementation of the new module (the export intelligence module) in the existing system.
- **Evaluation and Refinement**: Evaluation of the already implemented features to refine and enhance iteratively.
- **Future Iterations**: Subsequent iterations will focus on extending the system to include the export intelligence module until the system meets all the functional and non-functional requirements.

2.2.3. Practices Followed in Process Model

- **Prototyping**: Early iterations focus on developing functional prototypes of critical components (e.g. classification and segmentation modules) for validation and refinement.
- **Continuous Evaluation**: Each development iteration includes systemic evaluation of model performance, enabling data-driven improvements.
- **Risk Management**: Risks related to dataset quality, class imbalance and model generalization were identified and addressed through preprocessing, augmentation.
- **Incremental Testing**: Implemented modules were tested individually and jointly to ensure correct functionality and reliable interaction between classification and segmentation stages.

3. System Overview

The Export Edge AI project is an integrated agri-tech solution designed to transform the mango exporting business by reducing post-harvest loss and enhancing market competitiveness. The system combines automated quality control through Computer Vision with AI-driven market intelligence to bridge the physical quality measurement gap and the online market.

3.1. Functionality and Context

The **Export Edge AI** platform operates as a localized, web-based application that provides automated mango sorting and actionable market intelligence by integrating deep learning models and a multi-modal AI platform. It serves the following key use cases:

- **Real-time Quality Control:** The system utilizes computer vision to automatically sort and grade mangoes on a conveyor belt based on ripeness, size, and flaws.
- **Disease Analysis and Segmentation:** For quality assessment, it detects subtle defects and segments affected regions, moving beyond simple metrics.
- **Multi-modal Market Intelligence:** Exporters can key in or upload an image of a graded mango, and the AI generates a targeted market report with competitive pricing, current market demand trends, and country-related regulatory compliance information.

The system is designed to operate primarily on edge devices (like Raspberry Pi) for the Vision-Based Packhouse Automation System, with the web-based intelligence platform accessible to exporters from any device. The architecture is designed for end-to-end integration, linking the physical sorting directly with market intelligence.

3.2. Architectural Design

The Export Edge AI system employs a modular architecture, organized into an Edge-Optimized Layered Structure. This design is critical for ensuring low latency and performance despite deploying all heavy AI components locally on the limited resources of the edge device (Raspberry Pi). The design promotes scalability, simplifies maintenance, and ensures a clear sequential data flow. Below is an in-depth breakdown of the system's core layers and modules.

System Architecture Overview Diagram

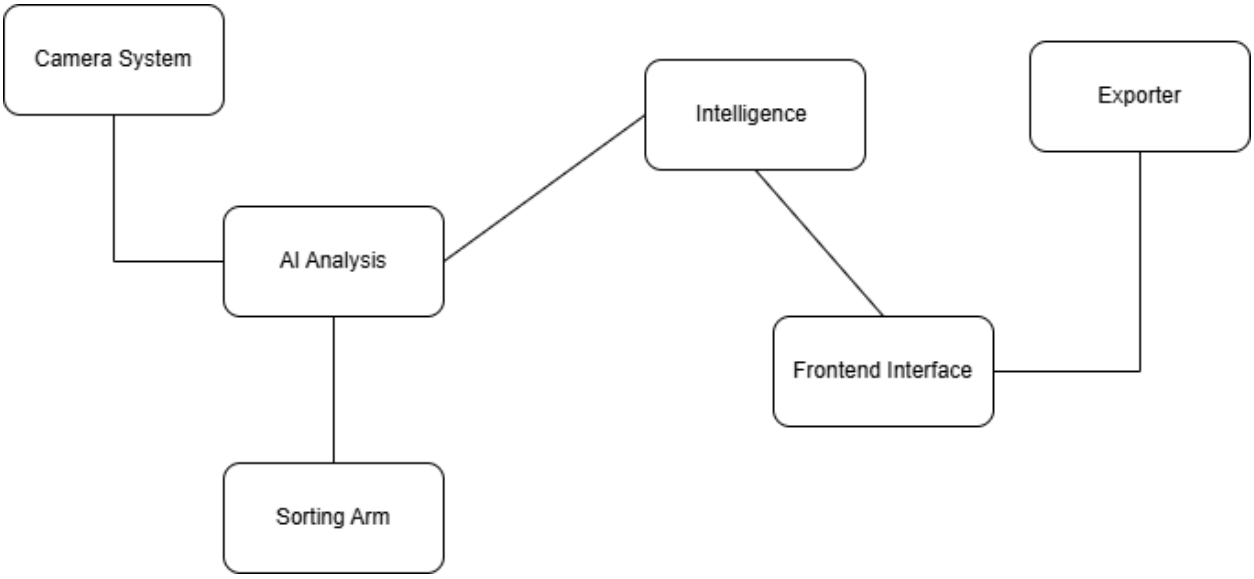


Figure 1: Box-and-Line Diagram of the Overall System Architecture

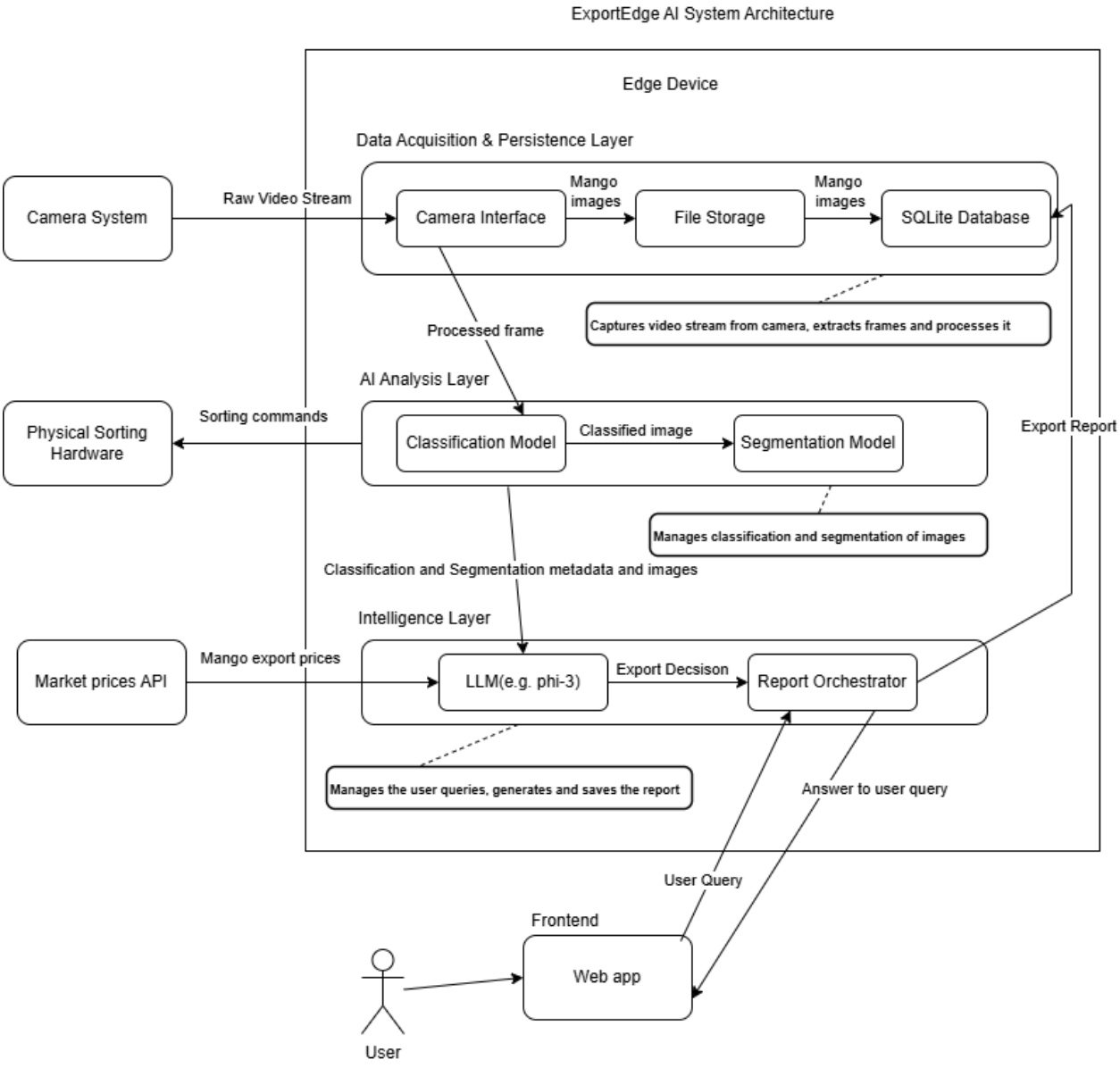


Figure 2: Architecture Diagram for ExportEdgeAI

3.2.1. Layer 1: Data Acquisition & Persistence Layer

- **Purpose**: Manages the physical hardware interface, secures the input data, and performs initial processing to optimize frames for the AI pipeline. This layer also manages all data persistence.
- **Technology**: Python with low-level libraries for hardware control (e.g., specific camera libraries, GPIO), and libraries like **OpenCV** for frame extraction and cleaning. Uses the **File System** for image storage and **SQLite** (or equivalent low-overhead local database) for structured report storage.
- **Interaction**: Receives raw video input from the camera, extracts frames, performs cleaning/resizing, and saves the image file locally with initial metadata.

3.2.2. Layer 2: AI Analysis (Processing Layer)

- **Purpose**: Executes the core Computer Vision tasks—the primary analysis for post-harvest quality control—at the edge.
- **Technology:** Uses high-performance Deep Learning frameworks such as **Tensor Flow Lite/PyTorch Mobile** for efficient inference of the Classification Model and the Segmentation Model on the edge device.
- **Interaction:** Receives a preprocessed frame from the Data Acquisition Layer. Executes the models sequentially: Classification runs first. Segmentation is executed for all mangoes. Temporarily stores selected image samples and metadata for batch processing.

3.2.3. Layer 3: Intelligence Layer

- **Purpose:** Serves as the application and business logic layer, orchestrating the system's workflow, executing the LLM, integrating market data, and generating high-value export reports.
- **Technology:** Python application using frameworks (e.g., **FastAPI/Flask**) for API communication, custom LLM models optimized for edge deployment, and **Request Libraries** for integrating **External Market Prices APIs**.
- **Interaction:** It has the following interactions:
    - **Automated Flow**: After a batch of images is classified, the LLM is triggered to generate the export report. Writes the resulting report data into a local database.
    - **User Flow**: Receives the user query from the frontend, processes them using the LLM and RAG knowledge base and returns contextual answer to the user.

3.2.4. Layer 4: Frontend Layer

- **Purpose:** Provides the user interface (UI) and acts as the presentation layer, delivering real-time feedback and the final intelligence reports to the exporter.
- **Technology:** Web-based framework (e.g., **React.js**) designed to be mobile-responsive for use in a packhouse environment.
- **Interaction:** Renders the real-time status of the sorting process. Sends user commands and QA queries to the Intelligence Layer and displays the generated reports and LLM responses.

3.3. Modular Collaboration

- **Sequential Data Flow**: Data moves strictly from layer1 to layer2 to layer 3. The intelligence layer manages the batching mechanism to ensure the LLM receives data only after the necessary number of mangoes have been processed by the AI analysis layer.
- **Data Persistence Loop**: The Intelligence Layer writes the valuable export reports into the Data Acquisition Layer's database, and the Frontend Layer reads this information via the Intelligence Layer for presentation.
- **User Interaction**: The frontend layer initiates user requests directly with the intelligence layer, which provides a conversational response using the local LLM.

3.4. Decomposition Rationale

The system is decomposed into modular components to:

- **Simplify Development and Maintenance**: Each layer has clear boundaries and responsibilities, which simplifies troubleshooting and allows us to work independently on hardware interfacing (Layer 1) versus market intelligence (Layer 3).
- **Manage Edge Constraints**: The layered structure facilitates resource scheduling; high-demand operations like CV (Layer 2) are completed before the LLM (Layer 3) is loaded and executed, preventing resource contention on the Raspberry Pi.
- **Promote Reusability:** The **AI Analysis Layer** components (Classification/Segmentation) are encapsulated, allowing them to be easily reused or adapted for sorting other agricultural products.

Explanation of Components:

- **Frontend Layer (Web interface):** Web-based framework application. Provides the user interface and presentation layer. It displays the export report, sends user queries to the intelligence layer.
- **Intelligence Layer**: Python application (FastAPI/Flask), the local LLM model and external market price APIs. It triggers the LLM after a batch is ready, writes the final export report to local database and answers user follow-up questions.
- **AI Analysis Layer**: Tensor flow Lite/Pytorch Mobile. It executes the real-time computer vision quality control. Classification runs first and then mangoes proceed to segmentation for detailed surface analysis.
- **Data Acquisition**: OpenCV and SQLite. It manages physical input and data persistence.

4. Design Models

The design models for ExportEdge AI are as follows:

4.1. Use Case Diagram

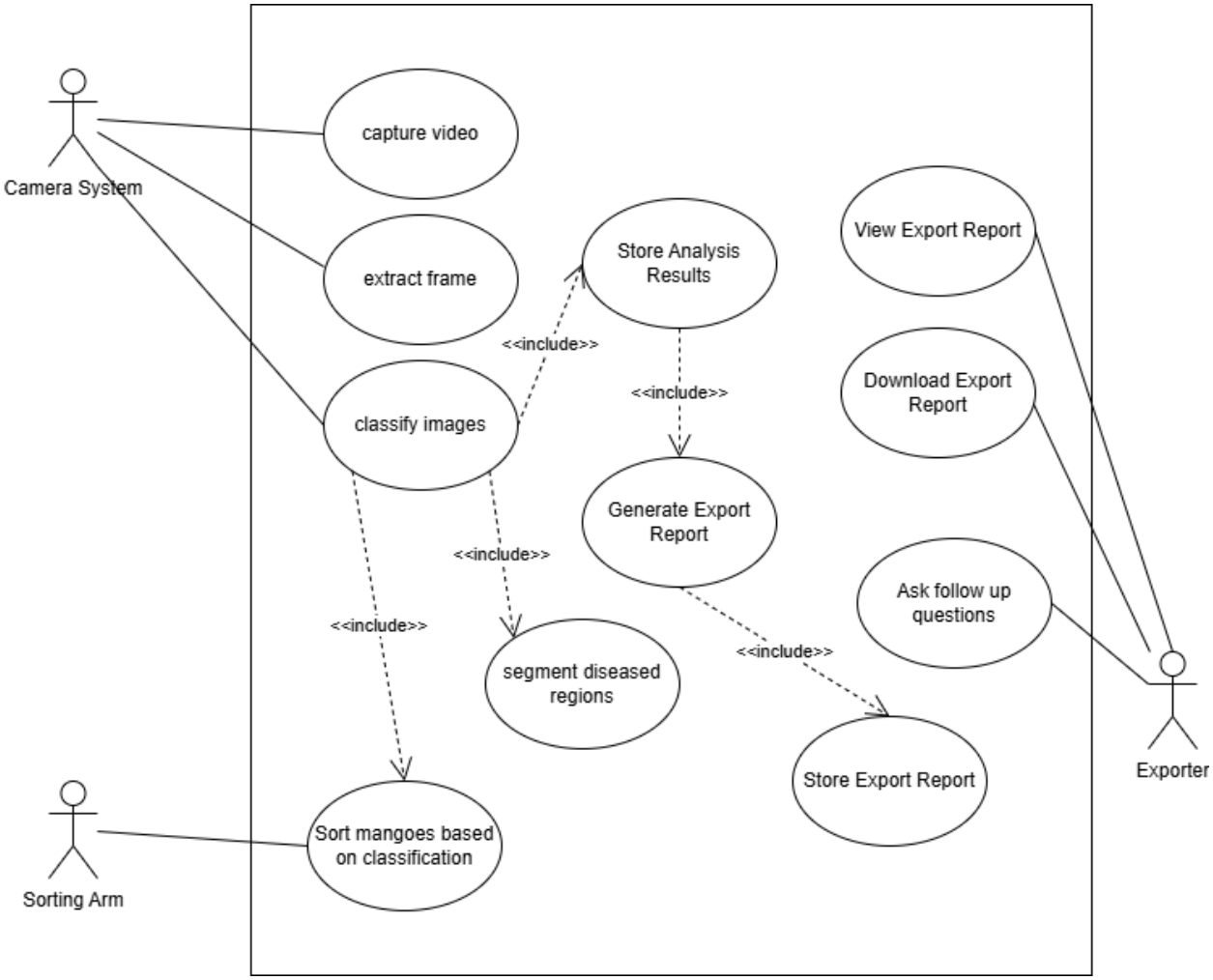


Figure 3: Use Case Diagram for ExportEdgeAI

Explanation:

- The Camera System initiates the automated quality assessment process by capturing live video and extracting frames of mangoes moving on the conveyor. These frames are classified using AI models, and mangoes undergo additional segmentation to localize defected regions. The classification results are stored for further processing.
- Based on the classification outcome, the Sorting Arm automatically separates healthy and diseased mangoes, enabling real-time physical sorting without human intervention.
- The Export Intelligence functions use the stored analysis results to generate detailed export reports. These reports are saved locally and serve as the foundation for decision-making.

4.2. Sequence Diagrams

4.2.1. Sequence Diagram: Automated Mango Processing Pipeline

Flow Description:

1. The camera system triggers the data acquisition layer to capture the live video stream.
2. The data acquisition layer extracts and pre-processes the frame extracted and sends the processed image to the AI analysis layer.
3. The AI analysis layer executes the classification process on the image.
4. Based on the classification result, the AI analysis layer immediately sends the command to sorting arm to sort the mango to respective bin (real-time).
5. The AI analysis layer then executes segmentation to recognize defected regions in the mango.
6. The analysis results (classification and segmentation metadata) are then sent to the Intelligence Layer.
7. The intelligence layer executes the generate export report process to generate the export report using LLM and market data.
8. Finally, the intelligence layer stores the generated export report in the database.

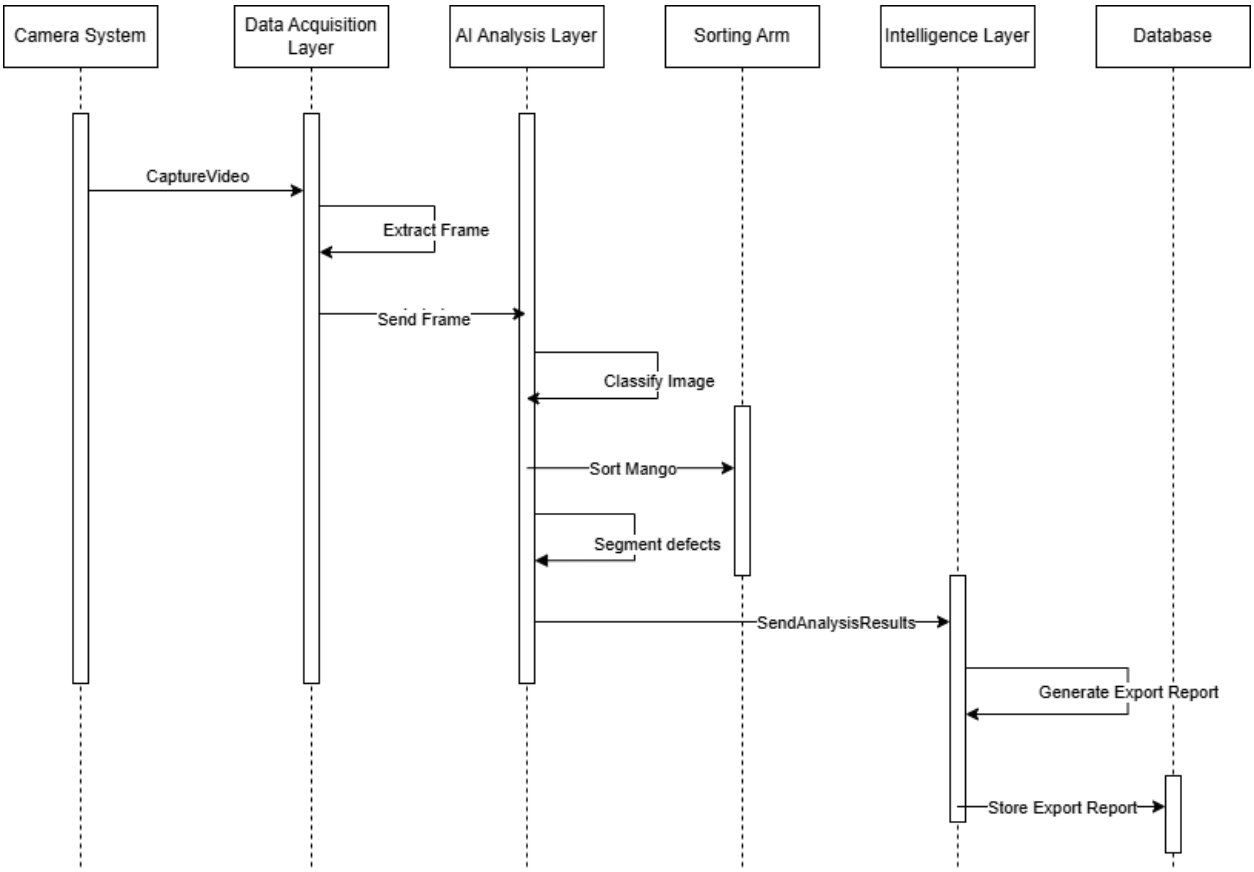


Figure 4: Sequence Diagram for Mango Processing Pipeline

4.2.2. Sequence Diagram: Frontend Report Retrieval

Flow Description:

1. The user initiates the process by sending a request to view the export report.
2. The frontend web app sends a message to the intelligence layer to get the export report.
3. The intelligence layer executes the getExportReport() method to send a request to the database.
4. The database retrieves the required data and returns the data back to the intelligence layer.
5. The intelligence layer creates the report in the required format and then sends the final export report back to the frontend.
6. The frontend renders and displays the export report to the user.

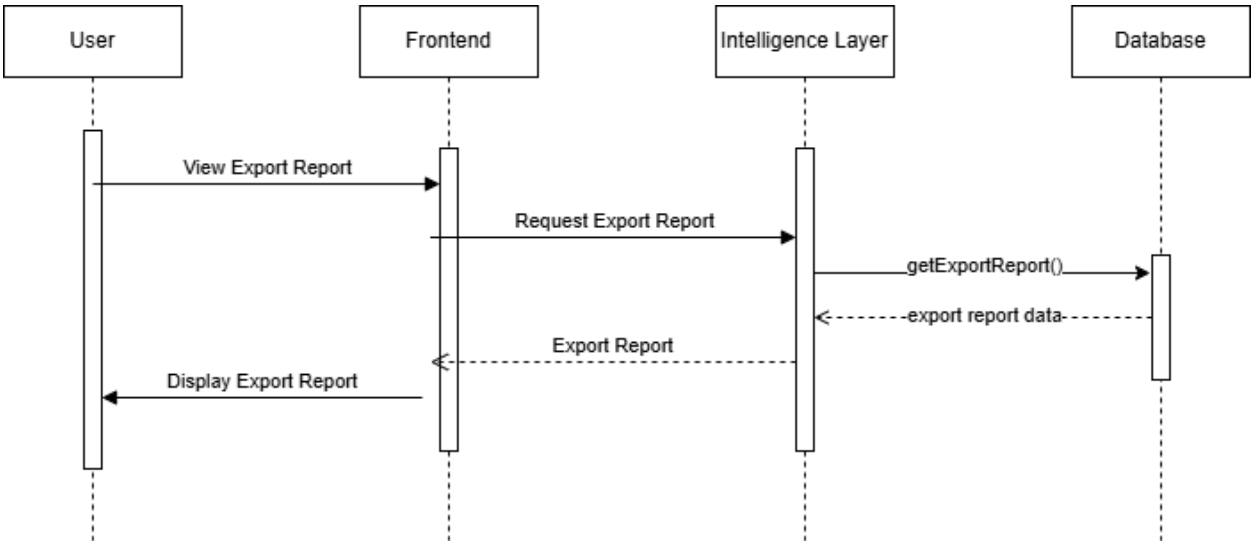


Figure 5: Sequence Diagram for Frontend Report Retrieval

4.2.3. Sequence Diagram: User Query and LLM Response

Flow Description:

1. The user initiates the process by submitting a query.
2. The frontend forwards the query to the intelligence layer.
3. The intelligence layer processes the query by initiating the RAG process and generating the final answer using the LLM.
4. The intelligence layer sends the generated answer back to the frontend.
5. The frontend then renders and shows the answer to the user.

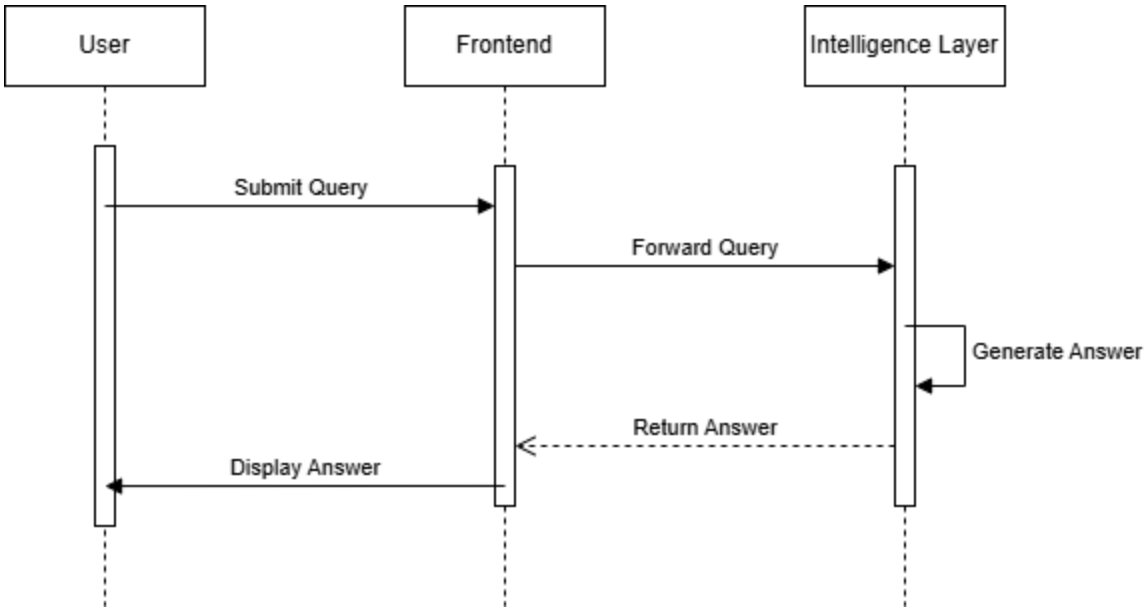


Figure 6: Sequence Diagram for user QA

4.3. Activity Diagrams

4.3.1. Activity Diagram: Real-time Mango processing and Sorting

This diagram illustrates the complete, continuous process from the moment the camera captures a frame to the physical sorting action and the subsequent batch management. It uses Swim lanes to partition responsibilities across the architectural layers and external hardware.

Workflow Description:

1. **Start & Acquisition:** The process begins by capturing the real-time video and extracting a stable frame.
2. **AI Analysis (Decision point):** The system runs the classification activity on the frame, which leads to segmentation of the image to mark the defected parts in the mango.
3. **Physical Action**: The control flow converges and immediately triggers the sorting arm to sort mango into respective bin.
4. **Intelligence & Archival**: The result is sent to the intelligence layer to be added to the batch.
5. **Batch check:** A second decision point checks if batch is ready or not?
    - If yes: The system executes the generate export and store export report.
    - If no: The batch waits and the process ends waiting for more processing of more mangoes.

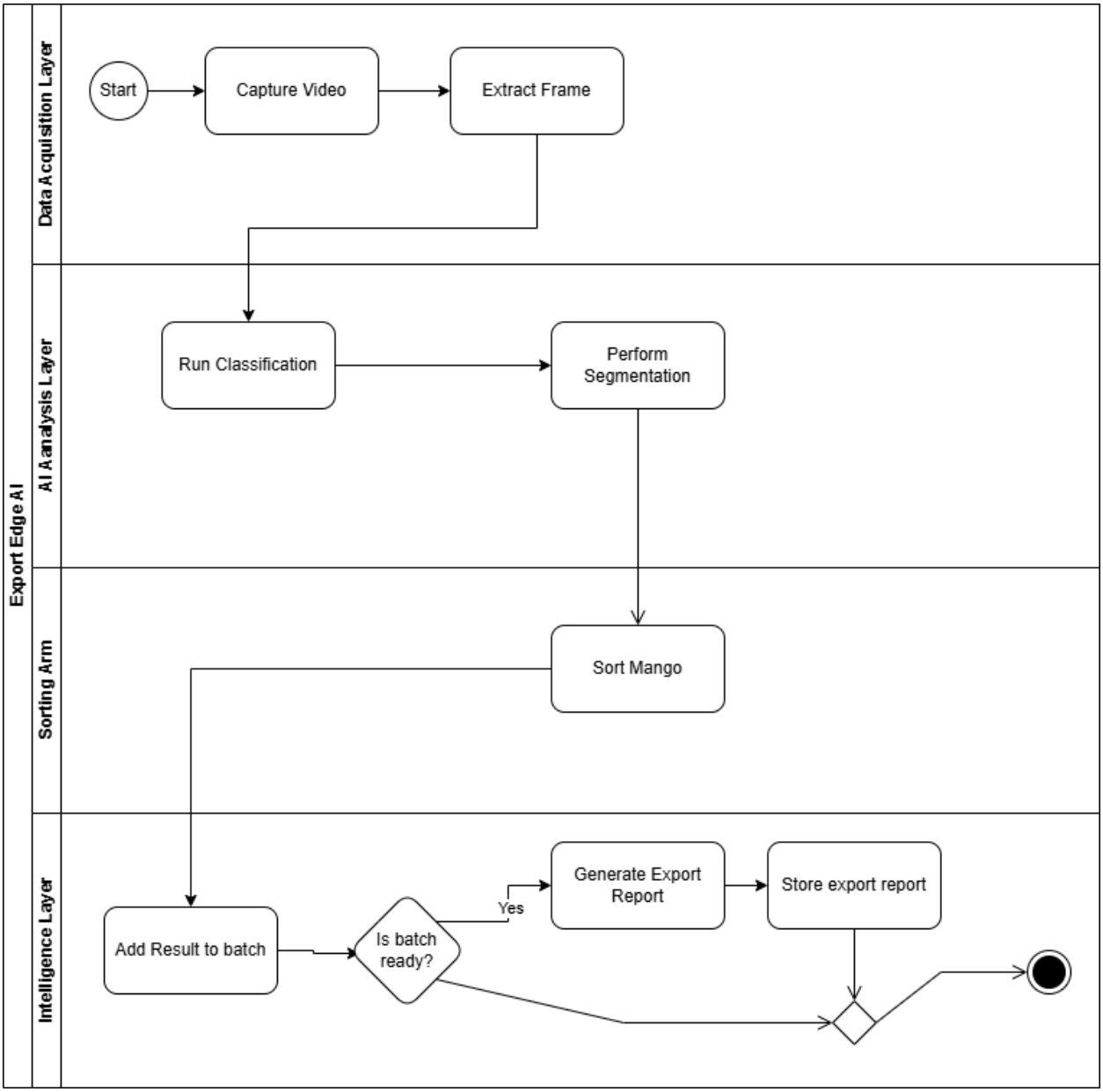


Figure 7: Activity Diagram for Mango Processing & Sorting

4.3.2. Activity Diagram: User Query Answering

This diagram illustrates the activity flow for a user submitting a natural language query, detailing the decision logic within the Intelligence Layer necessary for the Retrieval-Augmented Generation (RAG) process.

Workflow Description:

1. **Initiation:** The process starts when the user submits a query.
2. **Context check (decision point):** The query is passed to the intelligence layer, which reaches a decision diamond, context needed?

- If yes: The process executes the retrieve context activity (querying the vector db).
- If no: The process bypasses context retrieval.

3. **Answer generation:** The control flow converges and proceeds to generate answer activity, where the LLM uses the original query and any retrieved context to formulate a response.
4. **Completion:** The final answer is passed back to the user for displaying the answer and the flow terminates.

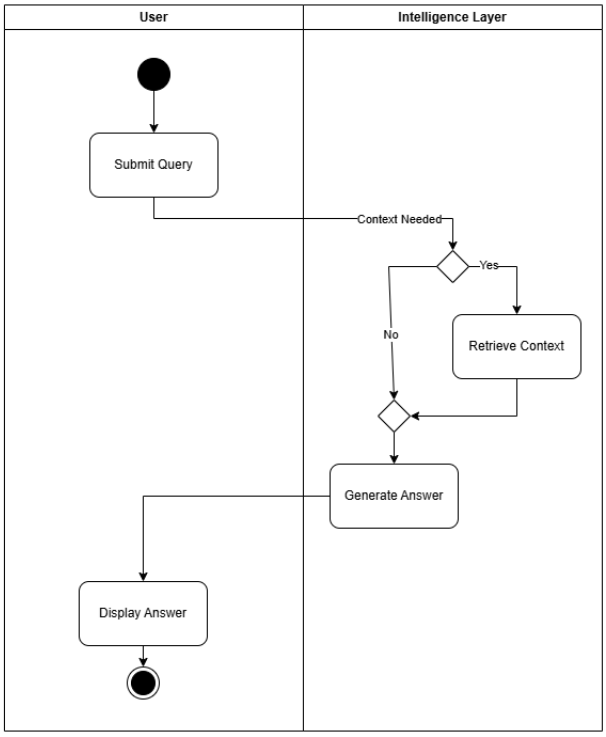


Figure 8: Activity Diagram for User QA

4.4. Data Flow Diagrams

4.4.1. Data Flow Diagram Level 0: Context Diagram

The Context Diagram (DFD Level 0) represents the entire **ExportEdge AI** system as a single process (P0) and identifies all external entities that interact with it. This diagram establishes the boundaries and scope of the entire system.

Description:

1. The camera feeds the continuous raw video stream into the system.

2. The market data external sources provide export price data necessary for decision making.
3. The system sends sorting commands to the sorting arm for real-time physical action.
4. The exporter interacts with the system by sending a user query and receiving a response (which includes LLM response and export report).

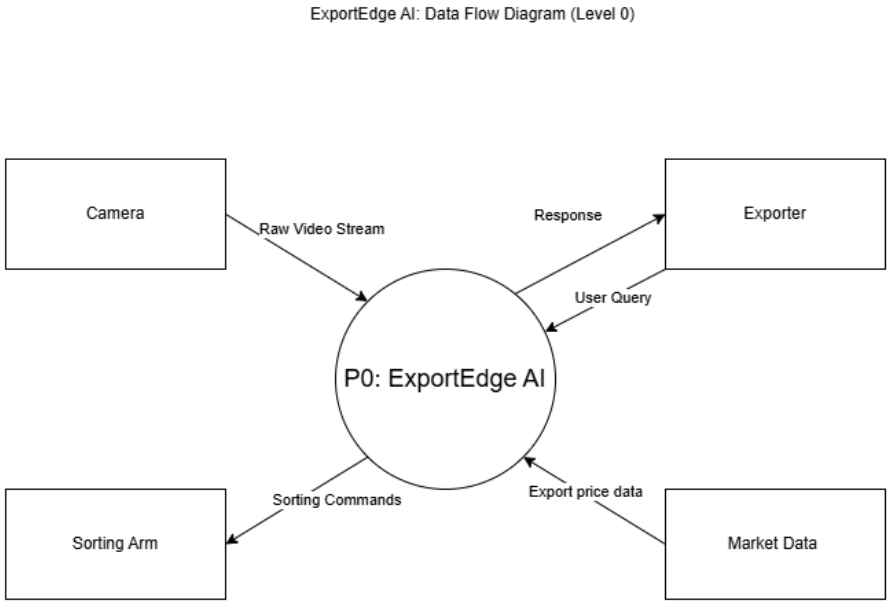


Figure 9: Level 0 (context diagram) Data Flow Diagram for ExportEdge AI

4.4.2. Data Flow Diagram Level 0: High Level System Decomposition

The DFD Level 0 decomposes the P0 process into the system's four major components (P1 to P4), which directly map to architectural layers, demonstrating the transformation of data step-by-step.

Description:

- **Data Stores:**
  - D1: Temporary Image Storage: used by P1 to store images.
  - D2: Export Report Database: persistent storage for final export reports. P3 writes to it and reads export report data from it.
  - D3: Vector Knowledge Database: A specialized database used to store high-dimensional vector embeddings of export reports and regulatory documents.

- **Key Flows:**
  - The process begins with P1: Capture & Process Mango Image data which prepares the input and passes Processed mango image to P2: Analyze mango quality. P2 transforms this into analytical output, passing classification and segmentation metadata to P3: Orchestrate and Generate Report.
  - During analysis, P2 issues a sorting command directly to external sorting arm.
  - P3 acts a control center, integrating the classification and segmentation metadata with external export price data received from the Market Data entity.
  - P3 maintains a crucial loop with the D2: export report database. It writes the finalized export report to D2 for archival and reads export report data back from D2 for answering user queries.
  - P3 forwards the complete export report and LLM based answers to P4: Present Report & Answer Queries layer. P4 is responsible for receiving the user queries and presenting the final information back.
  - Process P3 converts structured reports into embeddings. When a user submits a query via P4, P3 retrieves the relevant "query context" from D3 to generate an accurate LLM response.

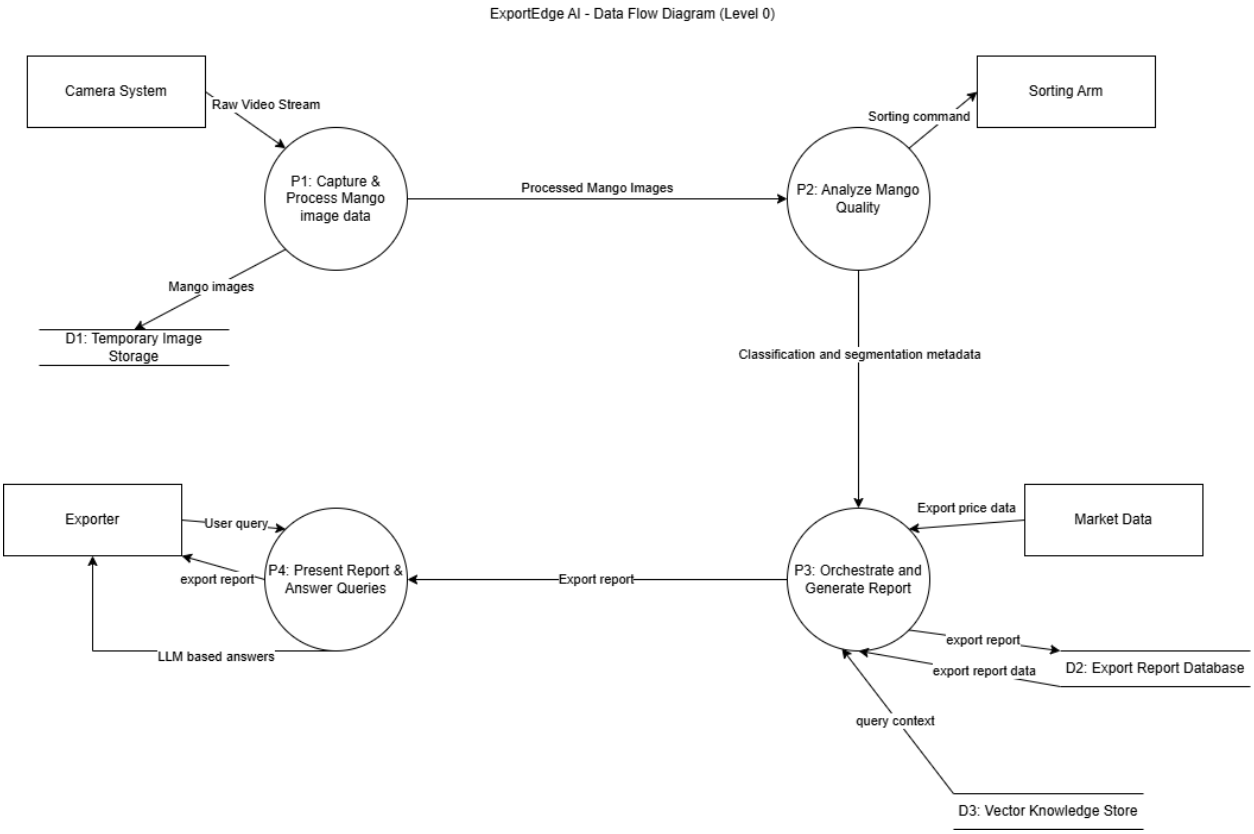


Figure 10: Level 0 Data Flow Diagram for ExportEdge AI

4.4.3. Level 1 DFD: Analyze Mango Quality

The DFD Level 1 for **Analyze Mango Quality** decomposes the primary analysis process into three functional sub-processes to show the internal logic of the computer vision pipeline. It illustrates the conditional relationship between classification and defect segmentation.

Description:

- P2.1: Classify Mango Image receives the processed frame from P1 and pulls necessary image data from D1: Temporary Image Storage to perform quality grading.
- If a disease is detected, the Classified Mango Image signal triggers P2.2: Segment Defects to calculate the specific area of infection.
- The results from classification models are passed to P2.3: Issue Sorting Command, which generates the physical Sorting Command for the Sorting Arm.

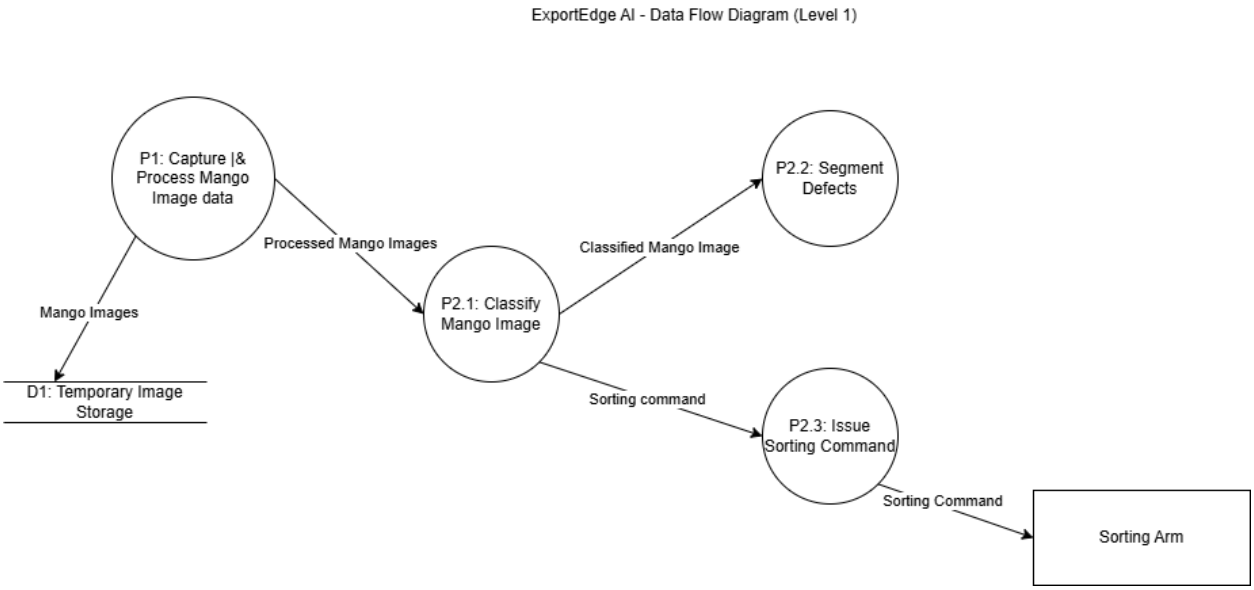


Figure 11: Level 1 Data Flow Diagram for Analyze Mango Quality Process

4.4.4. Level 1 DFD: Orchestrate and Generate Report

The DFD Level 1 for Orchestrate and Generate Report decomposes the intelligence layer into four sub-processes that handle data batching, market analysis, and the RAG-based LLM service. This diagram establishes the final persistence of intelligence into the system databases.

Description:

- P3.1: Manage Batch receives Classification & Segmentation metadata from P2 and buffers the data until a predefined batch is ready for analysis.
- P3.2: Market Analysis integrates the Batch data with Export price data from the external Market data entity to determine the most profitable export destinations.

- P3.3: LLM & RAG Service receives the structured export data and retrieves query context from D3: Vector knowledge store to generate a natural language report.
- P3.4: Save Report takes the finalized export report and persists it into D2: Export Report Database for long-term storage and user retrieval.

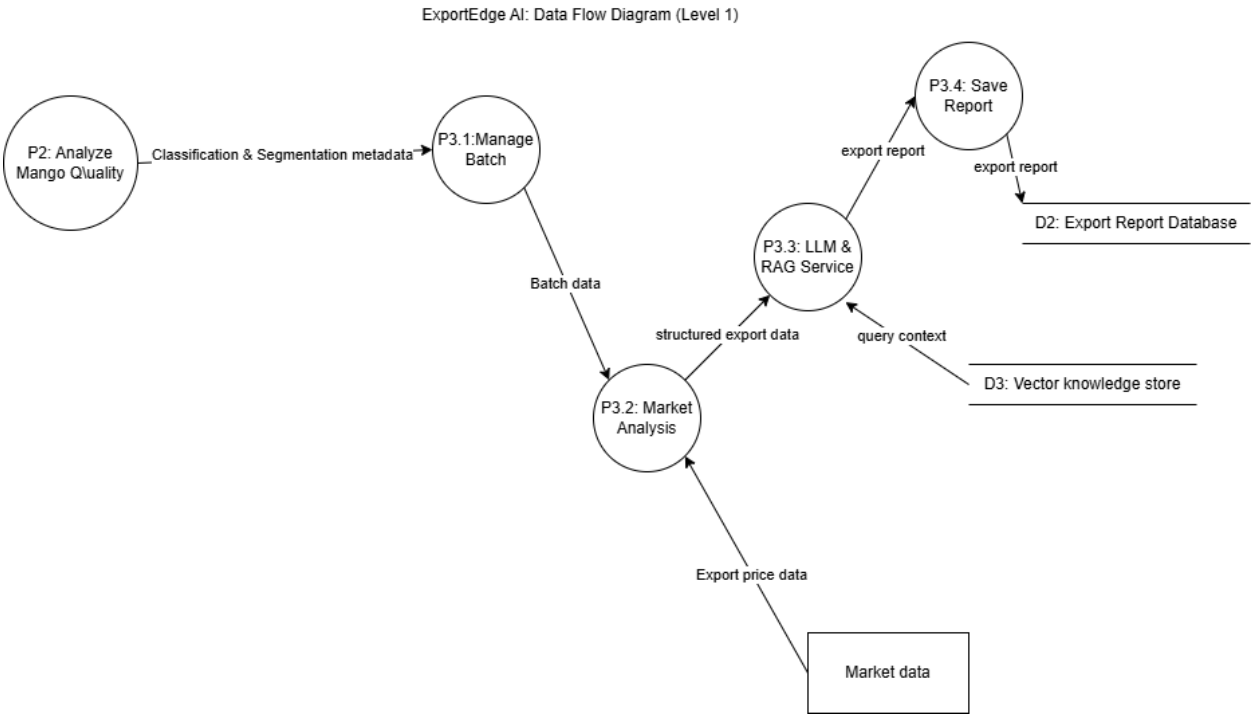


Figure 12: Level 1 Data Flow Diagram for Orchestrate & Generate Report Process

4.5. State Transition Diagrams

4.5.1. Packhouse Sorting Controller State Machine

This State Machine Diagram models the dynamic, real-time lifecycle of a mango as it is processed by the edge-deployed Computer Vision (CV) pipeline. It defines the high-speed operational states required to ensure accurate grading and physical sorting on the conveyor belt.

State and Transition Description:

- **Idle**: The default state where the system waits for a mango to enter the camera's field of view. A Motion detected event triggers the transition to processing.
- **Acquiring**: The camera interface captures the specific frame for analysis. Upon Frame captured, the system moves to AI inference.
- **Classifying**: The Mobile Net model runs to determine the mango's health and grade. Once classification completed, the system proceeds to detailed surface analysis.
- **Segmenting**: The DeepLab v3 model segments the mango surface to identify irregularities or defects. When the mask generated event occurs, the system moves to physical action.

- **Sorting**: The system sends a signal to the sorting arm. Once the Sorting Command sent event is complete, the controller returns to the Idle state for the next item.

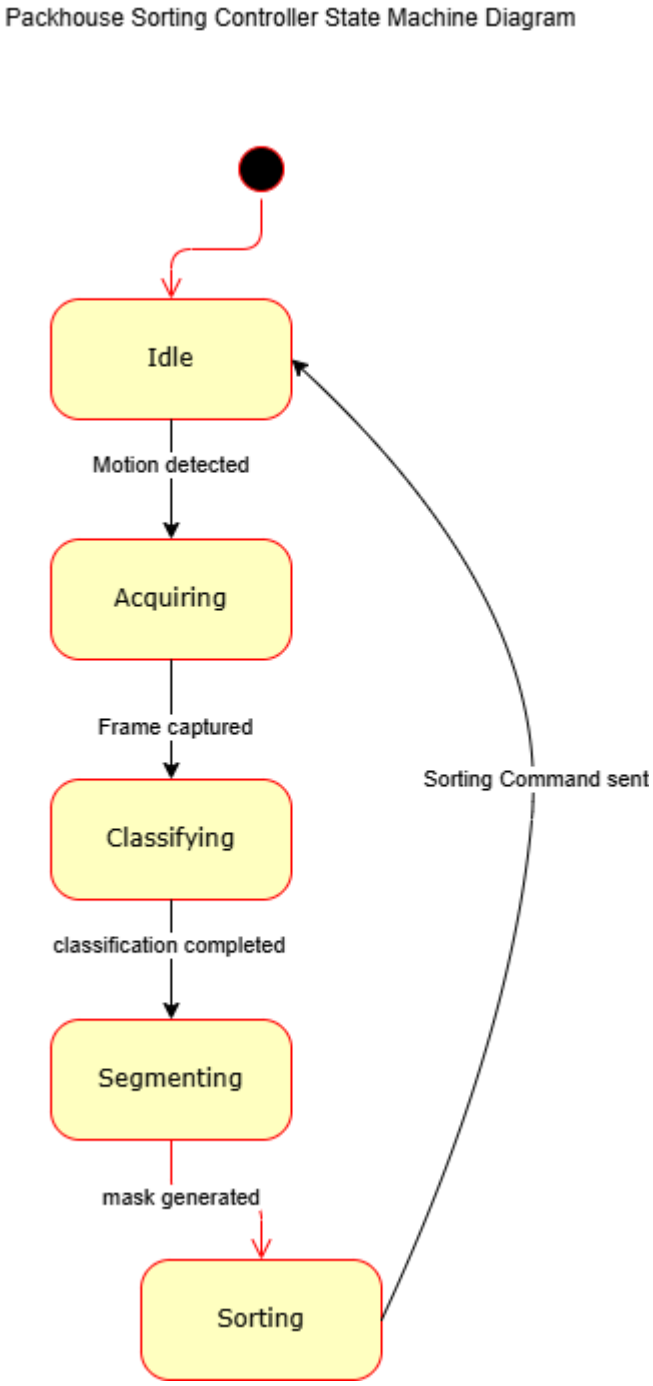


Figure 13: State Machine Diagram for Packhouse Sorting controller

4.5.2. Intelligence Orchestrator State Machine

This diagram models the logical states of the system as it orchestrates batch management, integrates market data, and utilizes the RAG framework to generate narrative export intelligence.

State and Transition Description:

- **Buffering:** The initial state where the system stays as it receives "Classification & Segmentation metadata" from the analysis layer. It remains here as long as isBatchReady == False.
- **Market Analyzing:** Triggered when isBatchReady == True. The system calculates optimal pricing and destination recommendations based on aggregated batch data.
- **Retrieving Context:** Upon the Market data fetched event, the orchestrator initiates the RAG process, querying the Vector Knowledge Store (D3) for regulatory and market compliance documents.
- **Generating Narrative:** Entered when context retrieved. The system is in this state while the LLM (e.g., Phi-3) is actively processing the retrieved context and batch data to produce the final narrative report.
- **Finalizing:** Triggered by response generation complete. The system completes the lifecycle by saving the report to the Export Report Database and presenting it to the exporter. Upon Save Successful, the orchestrator returns to the Buffering state.

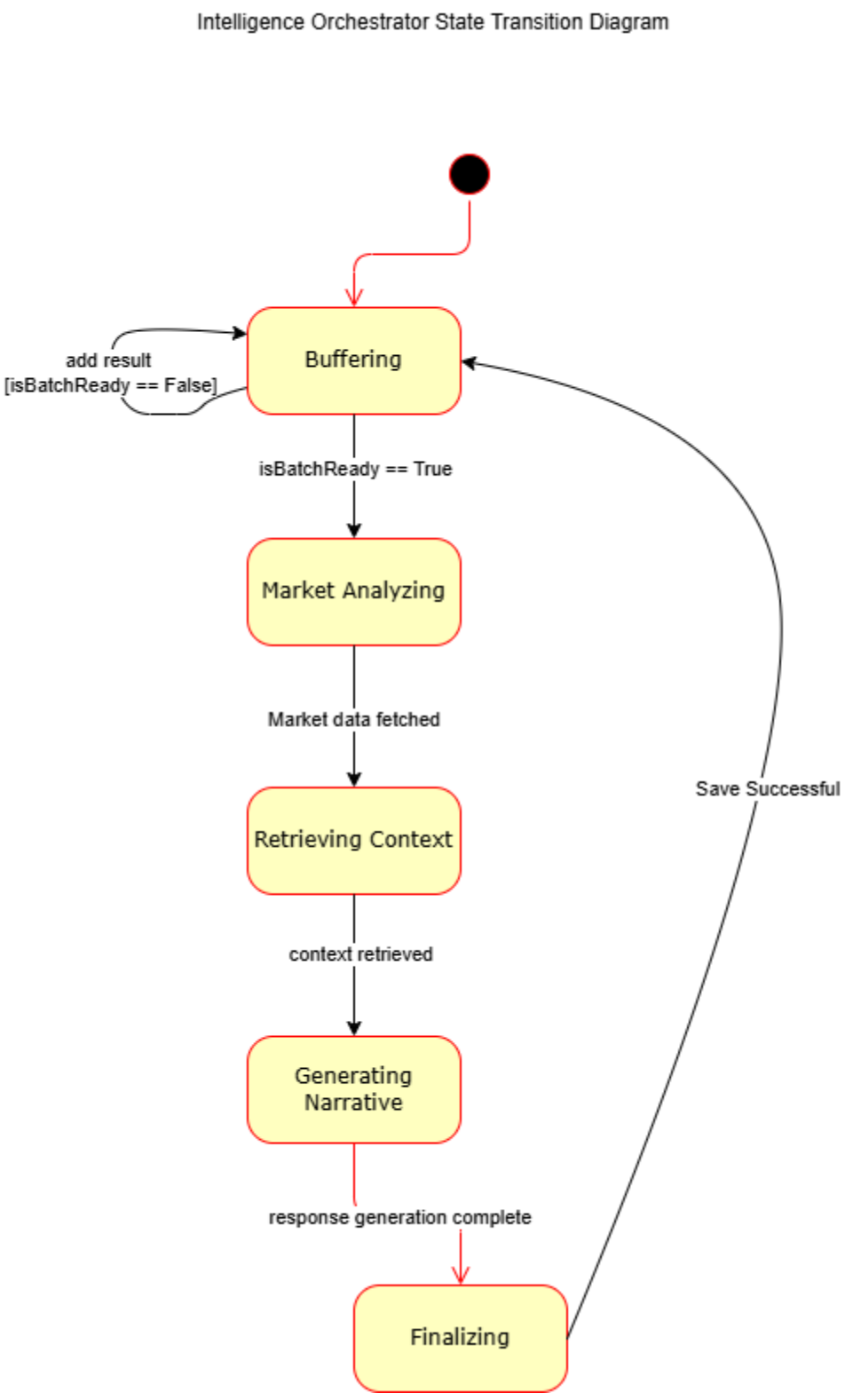


Figure 14: State Machine Diagram for Intelligence Orchestrator

4.6. Class Diagram

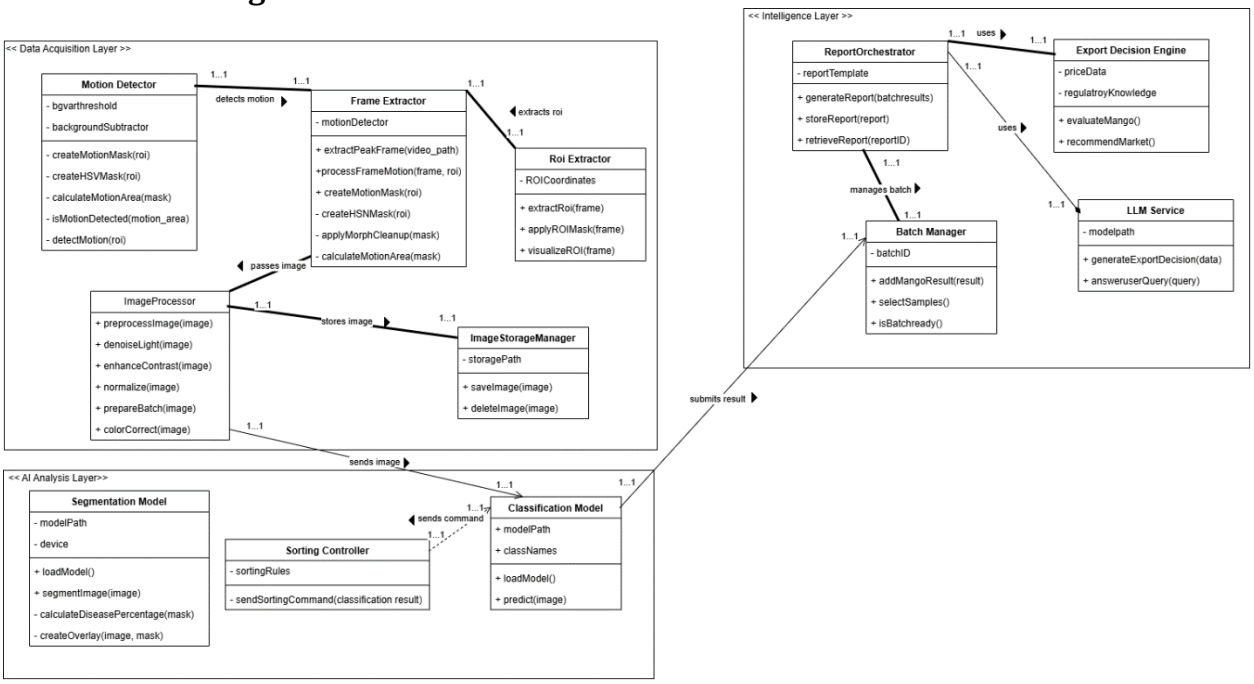


Figure 15: Class Diagram for ExportEdge AI

Explanation:

4.6.1. Data Acquisition Layer

This layer manages the real-time video stream input and image preparation.

- FrameExtractor is the primary controller of this layer.
  - It detects motion by interacting with motion Detector.
  - It extracts roi by utilizing the ROI Extractor.
- The image processor receives the frame from the frame extractor and passes images to the AI Analysis layer.
- The image processor stores images via the Image Storage Manager for temporary persistence.

4.6.2. AI Analysis Layer

This layer executes the core quality control and sorting logic.

- The classification Model receives the processed image and is the decision hub.
- It sends command to sorting controller for immediate physical action based on the result.
- The model's analytical output sends results to the intelligence layer.

4.6.3. Intelligence Layer

This layer acts as the control and orchestration center, managing batching, business intelligence and reporting.

- The ReportOrchestrator is the central class, utilizing other components to fulfill its reporting duties.
  - It manages batch by interacting with the BatchManager to track batch readiness.

    - It uses both the Export Decision Engine and LLM Service to generate comprehensive reports and answer queries.
- The batchmanager receives the analysis results via the classification model and it aggregates data until batch is ready.
- The ExportDecision engine uses external price data and regulatory knowledge to calculate optimal export markets.

5. Data Design

This section describes how the information domain of the ExportEdge AI system is transformed into structured data representations. It explains how system data is generated, processed, stored, and managed to support automated mango quality assessment and export intelligence generation.

The data design prioritizes **efficiency, scalability, and edge-device constraints**, ensuring that only high-value decision data is retained while intermediate processing data is discarded.

5.1. Overview of Data Organization

ExportEdge AI follows a **decision-centric data design approach**, where only finalized export intelligence is persistently stored. Intermediate data such as raw mango images and model inference outputs are treated as **temporary artifacts** and are not retained after report generation.

The system adopts a hybrid storage strategy:

- Temporary file system storage for raw mango images during processing
- Structured relational database storage for finalized export reports.

This design minimizes storage overhead, improves performance on edge devices and ensures long-term data usability.

This approach ensures high performance, modularity and efficient report rendering.

5.2. Data Lifecycle in ExportEdge AI

- Mango images are captured from the camera and stored temporarily in the local file system.
- Classification and segmentation models analyze images in memory.
- Export Intelligence is generated using regulatory and market data.
- A structured export report is created, embedding compressed visual references.
- The finalized export report is stored in the database.
- Raw mango images are deleted from the file system after report generation.

This lifecycle ensures that the system only retains meaningful, decision-level data.

5.3. Major Data Entities and Storage

5.3.1. Temporary File System Storage

The local file system is used during the active analysis phase.

Stored Temporarily:

- Raw mango image files captured from the camera.
- Batch-level intermediate metadata.

Once the export report is ready, these image files are removed to conserve storage and prevent redundancy.

5.3.2. Database Storage

A lightweight relational database (e.g. SQLite) is used to export intelligence results. The database contains self-contained export reports, enabling efficient retrieval and rendering. Only finalized export data is persisted.

5.4. Database Entities and Data Representation

5.4.1. ExportReport Entity

The ExportReport entity represents a complete export analysis generated for a batch of mangoes.

Purpose:

- Stores batch-level metadata.
- Serves as the parent entity for all mango-specific export decisions.

Attributes:

- report_id (Primary key)
- batch_id
- generated_timestamp
- overall_summary

5.4.2. ExportReportItem Entity

The ExportReportItem entity represents individual mango-level decisions within the report. Each record corresponds to a single mango and contains all information for visualization and decision-making.

Attributes:

- item_id (Primary key)
- report_id (Foreign key)
- image_snapshot (compressed visual reference)
- health_status
- predicted_disease
- recommended_export_country
- estimated_price_range
- regulatory_citations

The image_snapshot stores a compressed representation of the mango image, ensuring the report remains visually interpretable without relying on external image files.

5.5. Entity Relationship Diagram

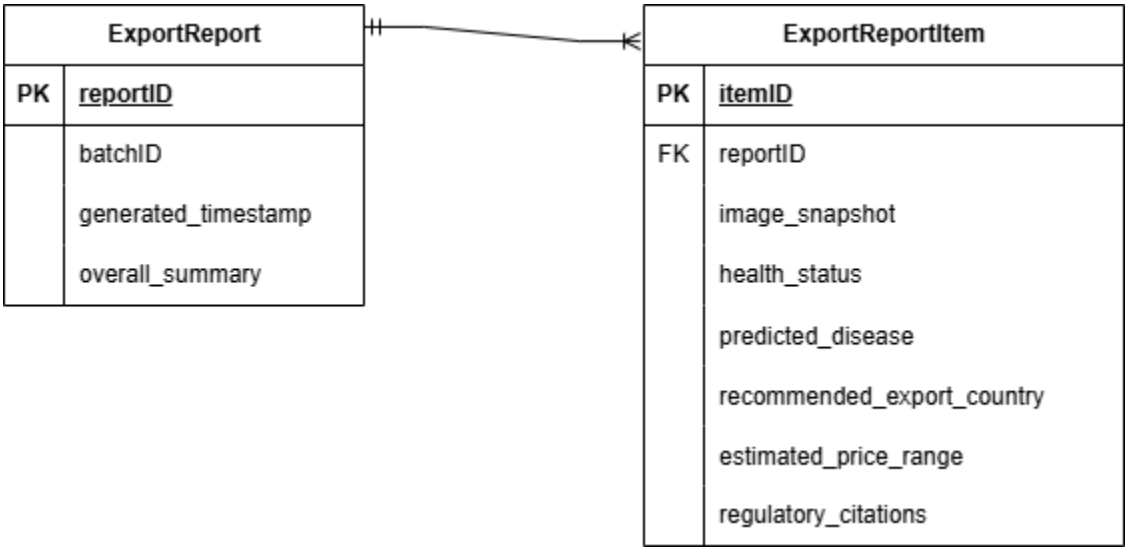


Figure 16: Entity Relationship Diagram for ExportReport Database

5.6. Data Dictionary

This section provides a structured data dictionary for the major system entities used in the ExportEdge AI platform. Since the system follows an object-oriented and modular design approach, the data dictionary focuses on the persistent database entities and their attributes. Temporary processing data and intermediate model outputs are excluded, as they are not stored beyond report generation.

5.6.1. ExportReport Entity

Description:

It represents a complete export report generated for a batch of mangoes processed by the system. This entity stores batch level metadata and serves as parent entity for all mango-level export decisions.

Attribute Name	Data Type	Description
report_id	Integer (primary key)	Unique identifier for the export report.
batch_id	Integer	Identifier for the mango batch processed.
generated_timestamp	DateTime	Timestamp which indicates when the report was generated.
overall_summary	Text	High-level summary describing batch quality and export suitability.

5.6.2. ExportReportItem Entity

Description:

It represents an individual-mango level export decision in the export report. Each record corresponds to a mango (visually embedded) selected for intelligence generation and includes all information required for visualization, decision making and report rendering.

Attribute Name	Data Type	Description
item_id	Integer (primary key)	Unique identifier for the report item.
report_id	Integer (foreign key)	References which report does item belong to.
image_snapshot	BLOB	Compresses visual representation of the mango image embedded in the report.
health_status	String	Classification result (e.g. healthy, diseased)
predicted_disease	String	Detected disease type (if any)
recommended_export_country	String	Suggested export destination based on quality and regulations.
estimated_price_range	String	Estimated market price for the mango
regulatory_citations	Text	References to regulatory or market policies used for decision making

5.6.3. Entity Relationships

The relationship between the entities is as follows:

- One exportReport can contain multiple ExportReportItem records.
- Each ExportReportItem belongs to exactly one ExportReport.

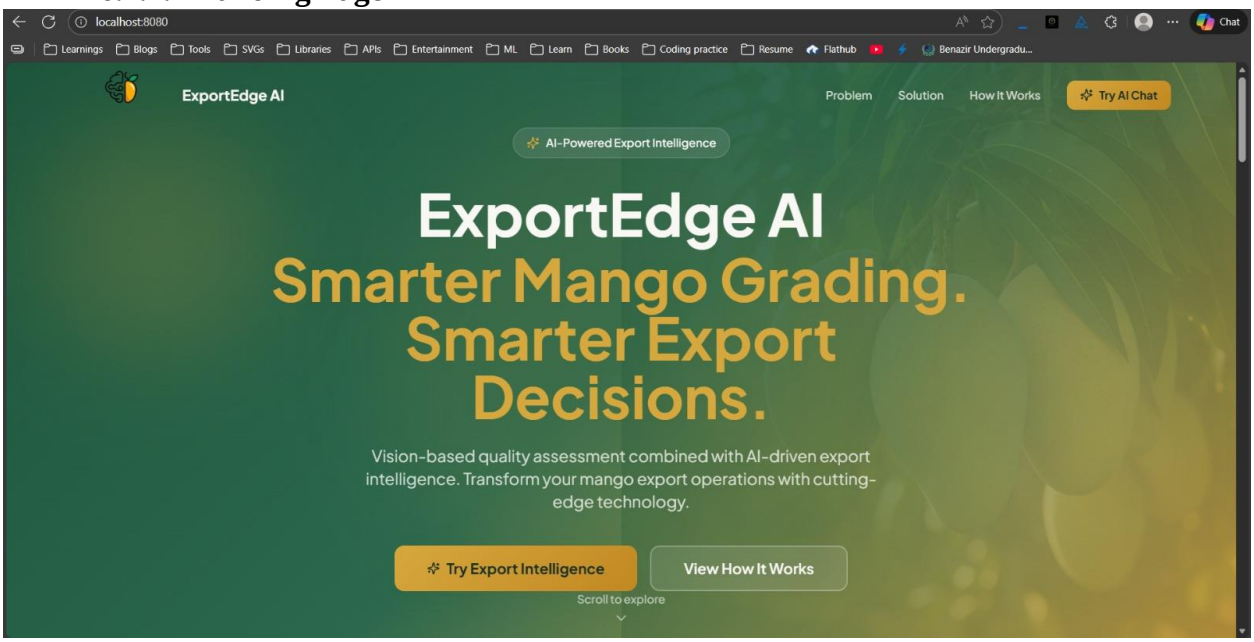
This one-to-many relationship enables efficient report retrieval and rendering.

6. User Interface Design

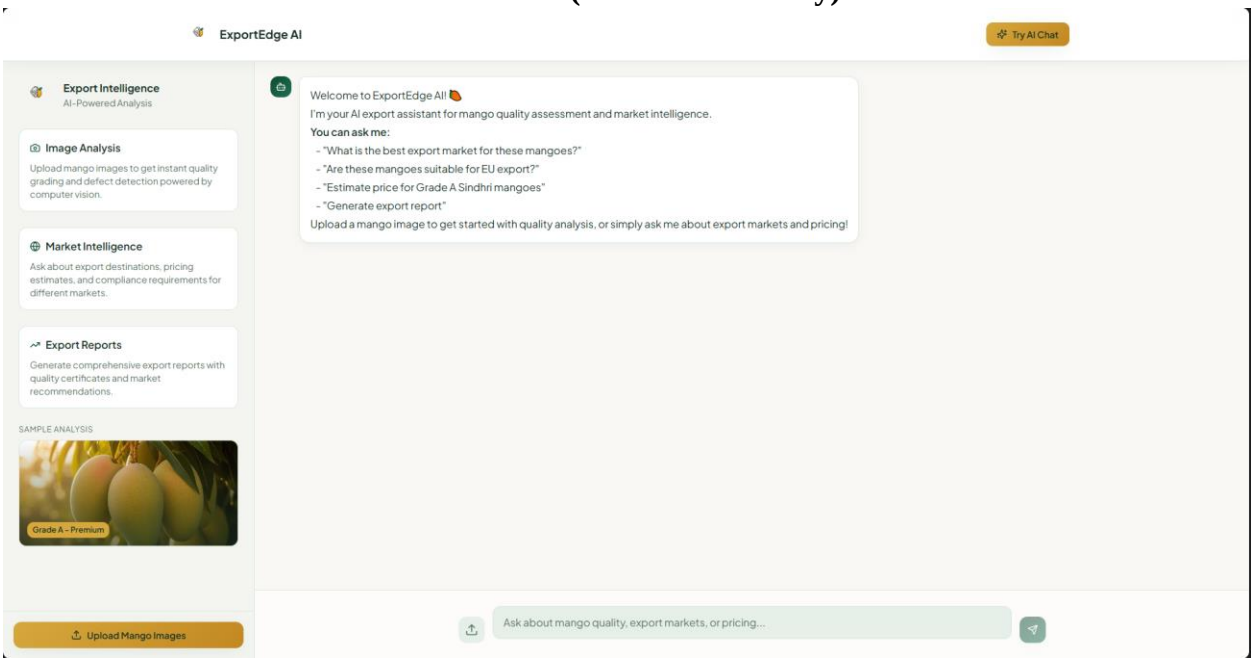
The ExportEdge AI system utilizes a two-stage, web-based interface designed for rapid access to both AI analysis and market intelligence. The design emphasizes a clear user journey, starting with a motivational landing page and transitioning into a functional, conversational dashboard.

6.1. Screen Images

6.1.1. Landing Page



6.1.2. Conversational Dashboard (Core functionality)



6.2. Screen Objects and Actions

This section details the primary interactive elements and the corresponding user actions that trigger the system's core features.

Screen Object	Description & Functionality	User Action & System Feedback
'Try export intelligence' button	The main call-to-action button that transitions the user from the high-level marketing page to the functional conversational dashboard	**Action:** User clicks the button. **Feedback:** The interface transitions to the conversational chatbot view.
Left Panel	The three non-interactive **div elements** on the left panel (Image Analysis, Market Intelligence, Export Reports) clearly **display the system's product features**.	**Feedback:** Serves as a continuous reference for the user, guiding the scope of possible queries and actions.
Chat Input Bar	The primary text input field for submitting natural language queries to the RAG system. This triggers the LLM Query workflow.	**Action:** User types a **User Query** (e.g., "What is the best export market?"). **Feedback:** System displays an **LLM based answer** (Response) after performing context retrieval and generation.
Submit Query Button	The button used to confirm and send the text entered in the chat input bar.	**Action:** User clicks the button after typing a query. **Feedback:** The query appears in the chat history, and the system begins processing the request.
Upload Images button	A button near the main chat input bar for uploading images, identical in function to the button in the bottom-left panel.	**Action:** User clicks to upload an image. **Feedback:** Initiates the AI analysis and displays results in the chat.